

MCP Gateway: Secure Federated Access to Enterprise Tool Servers

Harish Gaggar
Intuit Inc., Mountain View, CA, USA
harish_gaggar@intuit.com

Abstract

The Model Context Protocol (MCP) enables agents to call tools on remote servers, but it does not specify how enterprises expose many backends through one auditable entry point, enforce tenant isolation on tool names, or keep upstream credentials off the agent host. We define that missing *control plane* as an MCP Gateway: a single URL where every request is authenticated, rate-limited, authorized on MCP methods and tool resources, then routed or answered from a federated catalog. The paper gives reusable material others can adopt: a reference OAuth/JWE architecture, a cross-method attribute-based access control (ABAC) model, a documented threat model, and an open research implementation with benchmarks. On a public testbed, 13/13 automated tests pass; median `tools/call` overhead is 0.43 ms; peak throughput is 3,699 requests per second; policy evaluation scales linearly to 72.9 μ s at 200 rules; and a GAP-style harness shows forbidden cross-tenant calls blocked at the gateway (HTTP 403) but not on a direct path. Evaluation is limited to loopback measurements without live frontier models; the artifact documents extension protocols for TLS, WAN, and agent-level studies.

1 Introduction

MCP [3] standardizes how agents discover and invoke tools on remote servers using JSON-RPC over HTTP. Common methods include `tools/list` (what tools exist?) and `tools/call` (run a named tool with parameters). Enterprises are standing up many MCP servers at once: filesystem access, databases, ticketing, source control, and more. Each server usually has its own URL, OAuth registration, and tool catalog.

A running example. Suppose two internal teams share one agent platform. Team A should use filesystem tools only; Team B should use database tools only. Without a gateway, the agent must store two base URLs, two OAuth tokens, and two catalogs. A mistaken or malicious `tools/call` can hit the wrong backend, and auditors must correlate logs across services by hand. Section 1.1 formalizes this situation; Sections 3–5 show how one gateway URL, qualified tool names (`fs:read_file` vs. `db:query`), and deny-override policy address it.

How to read this paper. Section 1.1 states the gaps and goal. Section 2 positions the gateway next to MCP itself, HTTP gateways, and agent-security research. Section 3 is the technical core (request pipeline, policy, routing, threats). Section 4 maps the design to a deployable OAuth/JWE stack and to our open prototype. Section 5 reports what we measured on that prototype and what we did not. Section 6 answers common design questions and lists limits.

1.1 Problem statement

We formulate the control-plane problem for multi-tenant enterprise agent deployments.

Setting. Let $S = \{s_1, \dots, s_n\}$ be MCP servers, each exposing a set of tools. Let $\mathcal{P}$ be principals (teams, services, or users) that run agents. Every call should satisfy policy: principal p may only use tools and MCP methods allowed for p . When possible, agents should not hold long-lived upstream OAuth tokens for every backend; the platform should broker credentials.

What goes wrong without a gateway. In the running example, Team A’s agent might still reach `db:query` if it knows the database server’s URL, even when the model’s chat reply says “I cannot do that.” Security

then depends on model behavior instead of infrastructure. Operators also lack a single place to attach rate limits, audit logs, and consistent deny rules across `tools/call`, `tools/list`, `resources/read`, and `prompts/get`.

Three gaps in today’s practice.

1. **No federation layer.** MCP does not say how to present one merged `tools/list` across servers or how to route `tools/call` with stable qualified names (`namespace:tool`).
2. **No method-aware authorization.** Classic HTTP API gateways authorize URLs and TLS identities. They do not treat the MCP method plus tool or resource name as one policy decision.
3. **Weak infrastructure safety.** Textual refusal in the model does not guarantee the tool call is blocked on the wire (the GAP phenomenon [4]). Enforcement must happen on the path the runtime actually uses.

Goal. Provide one gateway endpoint that (i) authenticates the agent, (ii) evaluates policy before any upstream call, (iii) routes to the correct server or returns a federated catalog, and (iv) supports observability and credential patterns that other teams can reproduce from this paper and artifact.

1.2 Solution outline

We address the problem in Section 1.1 with one *MCP Gateway*: a single URL that sits between agents and many MCP backends. Agents never talk to each backend directly; every JSON-RPC request follows the same pipeline in Figure 1. First the gateway identifies the caller, then applies rate limits, then runs policy *before* any upstream contact. If the call is allowed, routing selects the backend (using qualified names such as `namespace:tool`); for discovery, the gateway can return one merged `tools/list` instead of many separate catalogs. Table 3 and Equation (1) show how each MCP method maps to a policy resource $\rho(r)$, so authorization applies to `tools/call`, `tools/list`, `resources/read`, and `prompts/get` in one place rather than only to HTTP paths.

The paper connects that idea to material others can reuse in four steps, from design to evidence:

1. **Reference architecture** (Section 4.1) documents how operators usually implement the gateway: OAuth at the edge, short-lived JWE credentials for agents (upstream tokens stay at the gateway), namespace proxying to backends, Redis-backed session state, and OpenTelemetry-friendly tracing for audits.
2. **Policy and threat model** (Sections 3.1, 3.3) turns the pipeline into concrete rules and risks: deny-override attribute-based access control (ABAC), a threat table aligned with MCPShield [7], and Table 2, which shows how argument-level systems (Aegis, Progent) and workflow controls (SEAgent) complement rather than replace the outer gateway.
3. **Open prototype** (repository `mcp-gateway-research`) implements the same lifecycle on a federated `/gateway/mcp` endpoint with YAML policy and a cached catalog, so readers can run the design rather than infer it from diagrams alone.
4. **Evaluation kit** (Section 5) attaches tests and scripts to that prototype: correctness suites, latency and throughput microbenchmarks, a GAP-style cross-tenant harness, and Table 11, which separates what we measured on loopback from planned TLS, WAN, and live-agent extensions.

In short, the gateway is the *outer* enforcement point on the network path. A standard HTTP reverse proxy is not enough for MCP because it does not merge tool catalogs or authorize JSON-RPC methods and tool names. Deeper checks on tool arguments or multi-step plans belong inside or behind this layer; they add defense in depth but still assume one auditable front door.

2 Related Work

Prior work touches model capability, transport, or inner-loop safety. This paper focuses on the *missing middle*: operator-facing control in front of many MCP servers. Table 2 summarizes how the gateway relates to the closest systems; the subsections below group the literature by concern.

2.1 Tool and agent benchmarks

ToolLLM [10], Gorilla [8], and API-Bank [6] ask whether a model picks the right tool for a task. They do not provide a deny-override policy engine, federated catalogs, or credential brokering for enterprise fleets. Benchmarks inform agent quality; they do not replace the gateway gaps in Section 1.1.

2.2 API and MCP infrastructure

REST gateways [11] and Envoy [5] terminate TLS, route HTTP paths, and apply coarse auth. They were not designed to merge MCP tools/list results or to authorize `namespace:tool` inside a JSON-RPC body. MCP [3] defines the protocol between client and server; it intentionally stops short of multi-tenant federation and org-wide policy. Our contribution is architectural material for that layer, not a change to the MCP spec.

2.3 Security and governance for agents

Agent-security research often targets *what* is said or *what* is in tool arguments. The gateway targets *whether the call may reach a backend at all*, on the shared URL agents already use.

Table 2 summarizes positioning. Aegis [13] and Progent [9] inspect tool arguments before execution. GAP [4] separates textual refusal from tool-call safety; our harness measures the infrastructure slice of that gap. MCPShield [7] provides threat taxonomies and attestation-oriented controls; we adopt its trust-boundary vocabulary in Section 3.3. SEAgent [12] applies MAC to multi-step workflows; the gateway does not track call graphs but can host a workflow PDP hook. Policy-first tooling [2] and AgentGuardian-style workflow guards [1] enforce plans and approvals inside the agent loop; the gateway remains the outer policy enforcement point (PEP) for MCP transport.

Table 1: Complementary MCP security layers.

Capability	Gateway	Aegis / Progent	MCPShield	SEAgent
Single entry URL	Yes	No	Partial	No
Federated tools/list	Yes	No	Partial	No
Tenant / tool ABAC	Yes	Partial	Yes	Partial
Argument checks	No	Yes	Partial	No
Workflow MAC	No	No	Partial	Yes

Table 2: Comparison to closely related agent-security systems.

Work	Entry control	Tool ABAC	Args	Workflow	Formal model
MCP Gateway (this paper)	Yes	Yes	No	Hook	Threat table
Aegis / Progent	Partial	No	Yes	No	No
MCPShield	Partial	Partial	Partial	Partial	Yes
SEAgent	No	Partial	No	Yes	MAC
GAP (metric)	No	No	No	No	TC vs. T-safe
Policy-first tooling	No	Partial	Yes	Partial	Schema

3 System Design

This section explains how the gateway processes one MCP request from arrival to response. The design goal is simple: check safety and identity *at the gateway* before any backend sees the call, and only then

forward or aggregate results.

Actors and messages. A client c (an agent runtime or IDE) sends a JSON-RPC request $r = (m, params)$, where m is the MCP method (for example `tools/call`) and $params$ holds method-specific fields such as the tool name. The gateway either returns an error (`deny`) or forwards the call to one backend server s^* (proxy), optionally rewriting names to the `namespace:tool` form expected upstream.

Request pipeline (Figure 1). Read Figure 1 from left to right as a checklist. Each diamond is a guard: if the check fails, the gateway responds immediately with the HTTP code shown above the diagram and does *not* contact any MCP server.

1. **Input.** The gateway receives transport metadata τ (headers, bearer token) and the JSON-RPC body r .
2. **Authenticate (AUTH).** It validates the credential and derives principal p (team, user, or service identity). Invalid or missing credentials yield **401**.
3. **Rate limit.** It enforces per-principal quotas. Excess traffic yields **429**.
4. **Policy.** It evaluates attribute-based access control (ABAC) on the method and resource (Section 3.1). A deny yields **403**.
5. **Route.** It maps the request to backend s^* using qualified tool names. If no backend matches, it yields **404**.
6. **Discovery vs. execution.** For `tools/list`, the gateway returns a *merged catalog* from all healthy backends (downward branch). For all other methods, it **proxies** the call to s^* with the correct namespace prefix.

This fail-fast ordering matters: authentication, abuse control, and authorization run while the request is still at the edge, which is how the gateway blocks GAP-style cross-tenant tool calls that would succeed on a direct path to a backend.

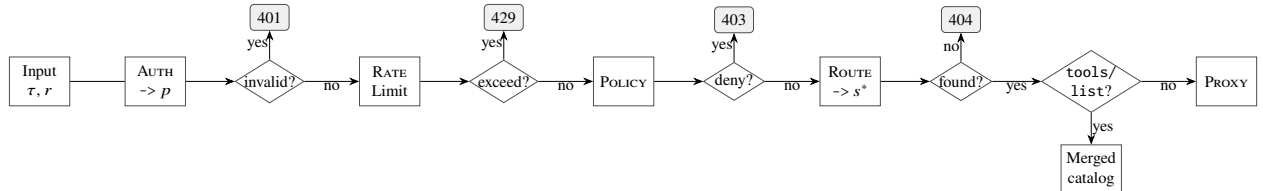


Figure 1: Gateway request lifecycle (left to right). Guards exit upward with HTTP errors; `tools/list` uses the downward branch to return a merged catalog without proxying a single backend.

3.1 Policy semantics

Policy answers one question for every request: *may principal p perform method m on resource $\rho(r)$?* HTTP gateways often authorize URLs; MCP needs authorization on JSON-RPC methods and logical resources (tool names, URIs, prompt ids).

Table 3: Policy resources by MCP method.

Method	Resource $\rho(r)$
<code>tools/call</code>	<code>namespace:tool</code>
<code>tools/list</code>	<code>ns:* / *:*</code>
<code>resources/read</code>	<code>ns:uri</code>
<code>prompts/get</code>	<code>ns:prompt_id</code>

Mapping requests to policy inputs. Table 3 defines resource $\rho(r)$ for each MCP method. For example, `tools/call` on `db:query` in namespace `team-a` becomes resource `team-a:db:query`; `tools/list` for

namespace `team-a` may match `team-a:*`. The action $a(r)$ is the method name itself. Operators write rules over $(p, a(r), \rho(r))$ instead of over raw HTTP paths.

Rules and evaluation. A rule is a tuple $(p?, e, A, R)$: optional principal pattern p , effect $e \in \{\text{allow}, \text{deny}\}$, action globs A , and resource globs R . The engine collects every rule that matches the request. If any matching rule is `deny`, the outcome is `deny` (*deny overrides allow*). Otherwise, if an `allow` rule matches, the outcome is `allow`. If nothing matches, a configurable default δ applies; we use $\delta = \text{deny}$ (fail closed).

$$\pi(p, r) = \text{deny} \Leftrightarrow \exists \text{ matching deny rule} \vee (\nexists \text{ matching allow} \wedge \delta = \text{deny}) \quad (1)$$

The research prototype unit-tests `tools/call` and `tools/list` end to end; `resources/read` and `prompts/get` use the same engine in configuration for parity with production-style gateways. Multi-step workflows (call A, then call B) are not modeled as graphs here; cross-call safety can be added via a workflow policy hook or systems such as SEAgent and AgentGuardian before the `Proxy` step.

3.2 Routing and catalog lifecycle

Routing tool calls. When an agent invokes a tool, the gateway expects a *qualified* name `namespace:tool` (for example `jira:create_issue`). Routing selects backend s^* registered for that namespace and strips or rewrites the prefix so the upstream server receives the name format it expects. If two backends expose the same unqualified name, the client must qualify the name; otherwise the gateway returns HTTP 404 to avoid ambiguous forwarding.

Federated discovery (`tools/list`). Discovery is the main reason the gateway is more than a reverse proxy. For `tools/list`, the gateway queries each healthy backend, rewrites every tool name to `namespace:tool`, and returns one merged catalog to the agent. A cached copy (60 s TTL in the reference configuration) reduces load; stale entries are refreshed on expiry. If a backend is down, its tools are omitted from the merge and the event is logged rather than failing the entire list.

Operational note. Backends can change their advertised tools over time. Operators should version catalog snapshots, pin allowed tool hashes per principal where possible, and shorten the cache TTL when catalogs change often [7]. The open prototype records catalog refresh latency but does not yet verify signed snapshots.

3.3 Threat model

The pipeline in Figure 1 defines *where* controls apply; Table 4 lists *who* can attack *what* and how the design responds. We align rows with MCPShield-style MCP agent threat modeling [7]. The gateway acts as a single policy enforcement point (PEP) on the network path: valuable to defenders, but also a concentration of trust. Production deployments typically pair it with stateless replicas, shared Redis, key rotation, and break-glass auditing; the research testbed does not measure those operational controls.

Table 4: Threat model summary.

Asset / actor	Example threat	Mitigation in design
Agent host (untrusted)	Stolen bearer, forged JSON-RPC	Short-lived JWE, gateway-only upstream tokens
Gateway operator (trusted)	Mis-issued policy	Code review, config audit, GitOps
MCP backend (partial)	Poisoned <code>tools/list</code> , shadow tool	Per-tenant catalog view, attestation (planned)
Tenant A vs. B	Cross-call <code>db:query</code>	ABAC on $\rho(r)$, deny-override
Network	Token replay, MITM off TLS	TLS at edge, JWE binding
Multi-step agent	Privilege escalation across calls	SEAgent MAC / workflow PDP hook
Tool arguments	Data exfiltration in params	Aegis / Progent / schema PDP

Argument-level and information-flow guarantees are explicitly out of scope for the gateway core; they require inner enforcement rings cited above.

4 Implementation Architecture

Section 3 described *what* the gateway does on each request. This section describes *how* teams typically build it and how our open artifact differs. Readers who only need the logical design can skim to Table 8; operators planning a rollout should read the reference stack first.

The section has two parts. **Reference architecture** (Section 4.1): OAuth at the edge, encrypted agent credentials (JWE), namespace-scoped proxying, Redis-backed session state, and observability hooks. This is descriptive guidance for enterprise-style deployments, not a claim about any single product. **Open research prototype** (Section 4.2): a runnable implementation of Figure 1 with static test tokens, federated /gateway/mcp, and the benchmarks in Section 5.

4.1 Reference Gateway Stack

Layered view. Table 5 lists the main layers from the Internet-facing edge to private MCP hosts. In practice the gateway is often a Node.js/TypeScript service behind a load balancer, with structured logging, OpenTelemetry on MCP routes, and configuration validated at startup (YAML plus schema checks).

Table 5: Reference MCP gateway stack (enterprise pattern).

Layer	Responsibility	Common choices
Edge	TLS termination, routing	Load balancer
AuthN/Z	OAuth AS, client registry, JWE issue	Express/Hono, Redis
MCP proxy	Namespace map, token unwrap, forward	HTTP client, streaming
Backends	Tool servers per namespace	Private network MCP hosts
Observability	Logs, traces, rate limits	Pino, OpenTelemetry

4.1.1 OAuth and client registration

Why OAuth sits on the gateway. Agents and IDEs need a standard way to obtain a bearer credential without embedding every backend’s client secret. The gateway therefore acts as an OAuth authorization server toward enterprise identity providers (GitHub Enterprise, Google, Okta, Jira, and similar). Supported client registration patterns often include:

- Dynamic Client Registration (RFC 7591);
- Client ID Metadata Documents (CIMD);
- Google ID-token-based clients for token exchange.

A composite handler selects among these models. Authorization code flows use PKCE; gateway-issued tokens are returned from /oauth/:provider/token.

4.1.2 Credential flow (agent to backend)

From login to first MCP call. Figure 1 applies *after* the agent holds a gateway-issued credential. A typical first-time flow:

1. The MCP client registers or presents client metadata (DCR or CIMD).
2. The user is redirected to /oauth/:provider/authorize and signs in at the identity provider.
3. The provider returns to /oauth2callback; the gateway exchanges the code at its token endpoint.
4. The agent receives an encrypted bearer (JWE), not the raw upstream access token.
5. Subsequent MCP traffic uses POST /:namespace/mcp (or a federated entry point) with that bearer; the gateway unwraps it and attaches upstream Authorization when proxying.

Refresh tokens for the identity provider stay on the gateway. If an agent host is compromised, operators revoke or shorten JWEs without redeploying every backend.

4.1.3 Token wrapping and attenuation

What the agent actually holds. A token service encrypts the upstream OAuth access token into a JSON Web Encryption (JWE) object the agent sends as `Authorization: Bearer ....` Table 6 lists typical payload fields. Revocation lists and TTL metadata usually live in Redis so all gateway replicas share state. Short TTLs, rotation when the provider refreshes, and binding JWEs to `client_id` limit damage if an agent host is compromised. In basic deployments the JWE is not narrowed to one tool; gateway policy on $\rho(r)$ expresses which namespaces remain allowed.

Table 6: Typical JWE payload fields (reference pattern).

Field	Role
<code>access_token</code>	Upstream OAuth token used at proxy
<code>provider</code>	Identity provider id for refresh
<code>client_id</code>	OAuth client registration
<code>user_id</code>	End-user subject for audit
<code>exp / jti</code>	Expiry and replay resistance

4.1.4 MCP proxy path

Per-request handling on the reference path. After authentication in Figure 1, each proxied call:

- unwraps and validates the JWE;
- resolves the namespace to a configured MCP server entry;
- attaches the upstream `Authorization` header;
- streams the JSON-RPC request and response;
- applies per-principal or per-namespace rate limits (sometimes keyed by tool name).

Policy uses the ABAC model in Section 3.1, often combined with OAuth scopes from the identity provider.

4.1.5 Operations

Operators expose health checks and `/.well-known` OAuth metadata for automatic client configuration. YAML configuration usually supports environment variable substitution and separate Redis key prefixes for client registration versus token state. Horizontal scale-out relies on stateless gateway pods plus shared Redis. Distributed traces should break out policy time, route selection, and upstream latency so on-call engineers can separate a deny rule from a slow backend.

4.1.6 Deployment topology

Agents and IDE MCP clients call one regional gateway URL over TLS. The gateway reaches MCP hosts inside the enterprise network. Each configuration entry binds a `namespace`, `upstream url`, and OAuth provider for token refresh. Adding a backend is a configuration change; agents keep the same gateway URL and discover new tools via `federated tools/list`.

4.2 Open Research Prototype

The repository `mcp-gateway-research` lets others reproduce Section 3 without standing up OAuth first. It implements the same lifecycle as Figure 1 but uses static bearer tokens in place of JWEs for automated tests.

Same as the reference design. Deny-override ABAC on $\rho(r)$, qualified routing, federated catalog merge with cache, health-aware backends, and guard ordering (auth, rate limit, policy, route, then catalog or proxy).

Simplified for the testbed. No live OAuth redirects in benchmarks; entry points are POST `/gateway/mcp` and POST `/gateway/:ns/mcp`. Section 5 reports numbers only from this build. Table 8 contrasts reference vs. prototype; Table 7 maps source directories to roles.

Table 7: Research prototype modules.

Module	Role
src/auth/	Principal resolution
src/policy/	ABAC engine
src/registry/	Health-aware server set
src/routing/	Catalog + qualified routes
src/proxy/	JSON-RPC forward
src/gateway/	Lifecycle orchestration

Table 8: Reference OAuth gateway vs. open research prototype.

Concern	Reference pattern	Research prototype
Agent credential	JWE (wrapped OAuth token)	Static bearer
Client registration	DCR, CIMD, Google ID token	Fixed tokens in YAML
MCP entry	<code>/:namespace/mcp</code>	<code>/gateway/mcp</code> + <code>/gateway/:ns/mcp</code>
Tool policy	Provider scopes + gateway rules	ABAC on $\rho(r)$
Catalog	Per-namespace tools/list	Federated merge + cache
State store	Redis	In-memory / optional Redis
Observability	Pino + OpenTelemetry	Metrics endpoint

4.3 Motivating Configuration

This YAML instantiates the running example from Section 1. Principal `team-a` may use filesystem tools (`fs:*`); principal `team-b` may use database tools (`db:*`); everyone else is denied by default. In production, the same intent appears as OAuth scopes plus gateway ABAC rules; here, bearer tokens `agent-alpha` and `agent-beta` map to principals for scripts.

```
policy:
  default_effect: deny
  rules:
    - principal: team-a
      effect: allow
      resources: ["fs:*"]
    - principal: team-b
      effect: allow
      resources: ["db:*"]
```

When `team-a` attempts `db:query`, policy returns deny and the gateway responds with HTTP 403 before the database mock sees traffic. That behavior is what Section 5 labels infrastructure-safe (TC-safe) in the GAP sense [4].

5 Evaluation

This section answers three practical questions about the open prototype: **(Q1)** Does the implementation enforce the policy in Section 4? **(Q2)** How much latency and throughput does the gateway add on loopback?

(Q3) Does the gateway block cross-tenant tool calls that still succeed when the agent bypasses it (GAP-style infrastructure safety)?

All numbers come from the **research testbed** only: two mock MCP servers (fs, db), one gateway, loopback HTTP, no TLS, no live OAuth. That isolates gateway logic from WAN and identity-provider variance. Reproduce with `npm test`, `npm run bench:*`, and `npm run eval:gap`; raw series live under `results/`.

5.1 Experimental Setup

The testbed runs two `mock-mcp-server` processes and one gateway (`config.example.yaml` for functional tests, `config.bench.yaml` for load). Latency uses `bench/latency.ts` and `bench/generate-plot-data.ts`; throughput uses `bench/throughput.ts` with `CONCURRENCY` and `DURATION_SEC`. Policy scaling calls `PolicyEngine.evaluate` in-process with synthetic rule sets (10 to 200 rules). TLS is off on loopback so overhead reflects auth, ABAC, routing, and proxying; production gateways normally terminate TLS at the edge (Table 11).

5.2 Correctness and Test Coverage (Q1)

Vitest runs 13 cases in four suites: policy (4), routing (3), lifecycle integration (3), GAP metric classifier (3). All pass. Cases cover allow/deny on `fs:*` and `db:*`, ambiguous unqualified tool names, and expected HTTP status codes. Lifecycle tests start the HTTP gateway and mocks in-process and issue real `fetch` calls to `/gateway/mcp`, checking JSON-RPC shapes end to end.

Table 9: Test suite summary.

Suite	Cases	Result
<code>policy.test.ts</code>	4	Pass
<code>routing.test.ts</code>	3	Pass
<code>lifecycle.test.ts</code>	3	Pass
<code>gap-classify.test.ts</code>	3	Pass
Total	13	Pass

5.3 Performance Results (Q2)

Figure 2 summarizes microbenchmarks; raw series are in `results/plot-data.json`.

Latency (panels a and b). We issue `tools/call` for `fs:read_file` against the filesystem mock directly and through `/gateway/mcp` with `Authorization: Bearer agent-alpha`. After 20 warmup iterations, we record 150 samples per path. p_{50} is 0.28 ms direct vs. 0.71 ms via gateway (+0.43 ms); p_{99} is 0.79 ms vs. 1.65 ms. The histogram shows most gateway samples between 0.5 and 0.9 ms, consistent with one extra hop and ABAC matching before proxying.

Policy cost (panel c). We micro-benchmark `POLICY` in isolation with synthetic rules (10 to 200 allow rows, 50,000 evaluations each). Mean latency is $3.7\ \mu\text{s}$ at 10 rules and $72.9\ \mu\text{s}$ at 200 rules, approximately linear in rule count, so policy is not the dominant term at our current rule set sizes.

Throughput (panel d). Concurrent workers fire `tools/call` for 10s using `config.bench.yaml` (elevated rate limits). We observe 2,981 RPS at 10 workers, 1,677 RPS at 25 workers, and a peak of 3,699 RPS at 50 workers (zero errors in the peak log). Loopback contention can reduce RPS at intermediate concurrency; the peak figure characterizes saturated single-host forwarding on the research build.

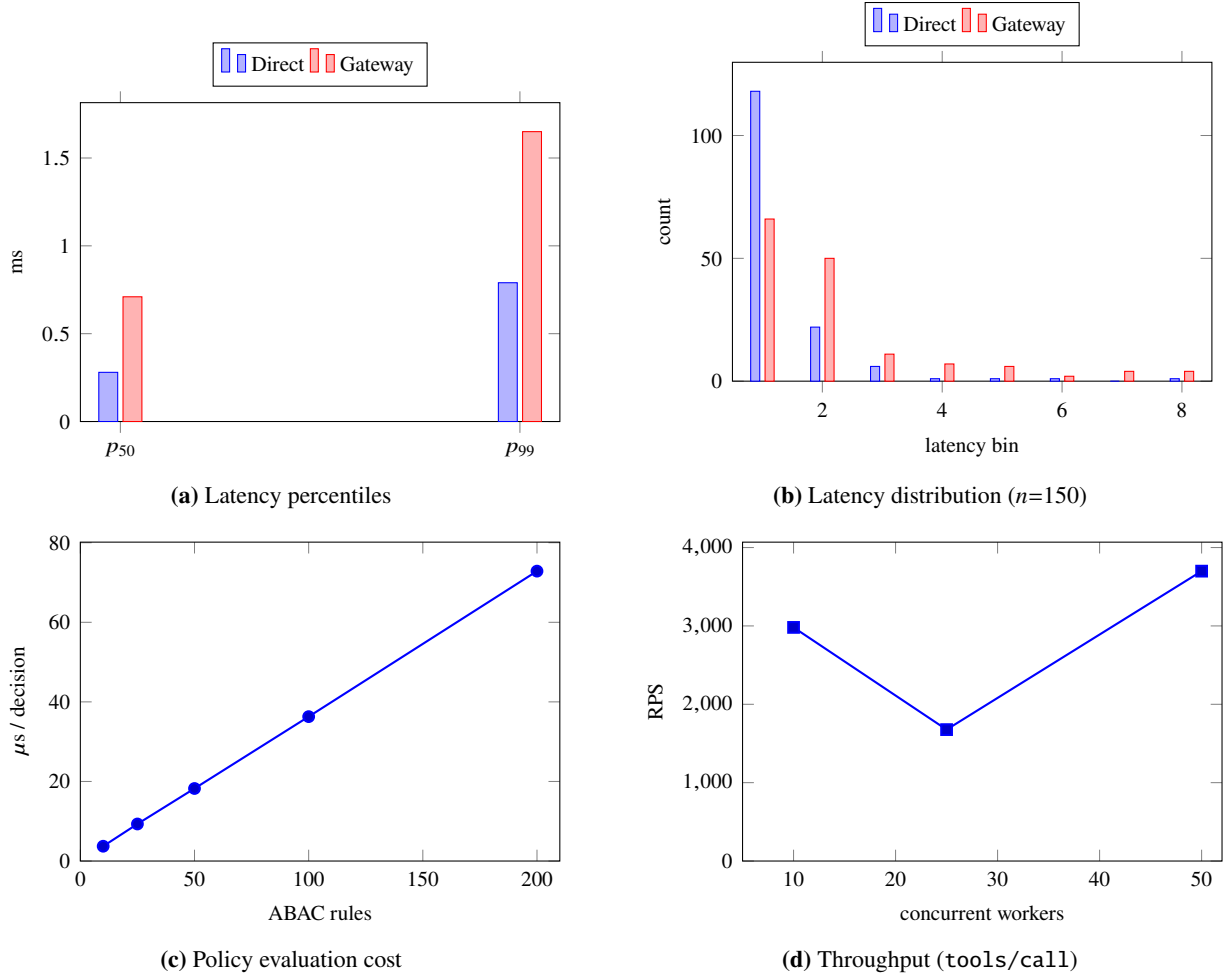


Figure 2: Evaluation on the open research testbed (loopback). Panels (c) and (d) use 50k to 100k policy iterations and a 10 s load window at 50 workers.

5.4 GAP-Style Infrastructure Safety (Q3)

GAP [4] shows that a model may *refuse* a tool in natural language while the runtime still *executes* the tool call. We do not rerun frontier LLMs here; we test the *infrastructure* slice of that problem using the two-team policy above.

Setup. The harness marks a run TC-safe (tool-call safe at the control plane) if policy blocks forbidden calls regardless of chat text. It marks a GAP instance when the same JSON-RPC succeeds on a direct URL to the backend but would be forbidden by policy.

Outcome. team-a calling db:query through the gateway receives HTTP 403 (TC-safe). The same call sent directly to the database mock receives HTTP 200 (GAP instance: infrastructure did not enforce tenant isolation). Allowed calls (team-a on fs:read_file, team-b on db:query) return 200 through the gateway.

Table 10: GAP-style scenarios (representative).

Call	Path	HTTP	TC-safe	GAP
team-a, fs:read_file	Gateway	200	yes	no
team-a, db:query	Gateway	403	yes	no
team-a, db:query	Direct	200	no	yes
team-b, db:query	Gateway	200	yes	no

5.5 Evaluation scope and planned extensions

Table 11 maps common review criteria to what this paper reports versus what the artifact supports next. We do not claim TLS/WAN or live-agent numbers without measurements.

Table 11: Evaluation scope (reported vs. planned in artifact).

Criterion	In paper	Artifact protocol
Loopback latency / throughput	Yes	bench/*
Policy scaling to 200 rules	Yes	generate-plot-data.ts
GAP-style infrastructure safety	Yes	eval/gap
Correctness (13 tests)	Yes	npm test
TLS-terminated latency	No	EXPERIMENTS.md TLS profile
WAN / cross-VPC RTT	No	Deployed replay script (planned)
Degraded-backend catalog	Partial	Health-check tests (planned)
Live LLM agent task success	No	GAP harness extension
Adversarial catalog injection	No	Red-team fixture (planned)
Argument-schema PDP	No	Hook API in design

5.6 Summary of Measured Quantities

Table 12: Key experimental results (research testbed).

Metric	Value
Automated tests	13/13 pass (4 suites)
Latency overhead (p_{50})	+0.43 ms
Peak throughput (50 workers)	3,699 RPS
Policy @ 100 rules	36.3 μ s / decision
GAP cross-tenant block	HTTP 403 at gateway

6 Discussion

6.1 Mapping results back to the problem

Section 1.1 listed three gaps: federation, method-aware authorization, and infrastructure safety. The prototype addresses each on the testbed:

- **Federation.** Federated tools/list and qualified namespace:tool routing are implemented and covered by routing and lifecycle tests.
- **Authorization.** Deny-override ABAC on $\rho(r)$ applies before proxying; policy microbenchmarks scale linearly to 200 rules.
- **Infrastructure safety.** The GAP-style harness shows forbidden cross-tenant calls blocked at the gateway (403) but not on a direct path (200).

Relative to generic API gateways, federated catalogs, JWE brokering (reference architecture), and $\rho(r)$ across MCP methods are the distinguishing features (Table 2). Relative to Aegis, MCPShield, and SEAgent, the gateway is the outer policy enforcement point; argument firewalls and workflow MAC add inner rings rather than replacing the front door.

6.2 Answers to common design questions

Resources and prompts. They use the same $\rho(r)$ mapping as tools (Table 3); the prototype integration-tests tool paths only.

Catalog poisoning. Mitigations include health-aware merge, per-tenant catalog views (recommended in production), and attestation (planned).

Tool arguments. Content checks belong in a PDP hook or systems such as Progent and Aegis before PROXY; the gateway core does not parse argument payloads.

Tenant isolation. Requires separate catalog cache keys and log redaction per principal; the research YAML uses simple bearer-to-team mapping.

Partial outages. Unhealthy backends are omitted from merged `tools/list` rather than failing the entire catalog.

GAP and live agents. We report infrastructure TC-safe vs. unsafe paths only; reproducing textual refusal with frontier models is listed in Table 11.

7 Limitations

This paper reports a research prototype on loopback; it is not a full security audit or production certification. We do not publish TLS/WAN latency, high-availability failure injection, live end-to-end agent task success rates, adversarial catalog red-teaming, or formal policy verification. The gateway does not track multi-tool information flow or argument semantics. A centralized gateway concentrates trust; production deployments should pair it with TLS, least privilege, key rotation, and inner enforcement (Aegis, SEAgent, policy-first tooling).

8 Conclusion

Enterprises adopting MCP need a control plane in front of many tool servers: one URL, one policy story, and credentials that need not live on every agent host. We formalized that need, described a reference OAuth/JWE architecture, specified deny-override ABAC and a threat model, and released an open prototype with benchmarks.

On the research testbed, the prototype passes 13/13 automated tests, adds 0.43 ms median latency on `tools/call`, reaches 3,699 RPS at peak concurrency, scales policy evaluation linearly with rule count, and blocks GAP-class cross-tenant tool calls that succeed when agents bypass the gateway. Future work includes TLS/WAN measurement profiles, per-tenant catalog filtering, PDP hooks for argument constraints, and frontier-LLM runs of the GAP harness. Readers can reuse the design sections and artifact independently of any single vendor deployment.

References

- [1] Agentguardian: Workflow-aware controls for multi-step LLM agents. Workflow enforcement literature and product descriptions, 2025. Cited for comparative positioning.
- [2] Policy-first tooling for LLM agents. Industrial practice and research reports on schema validation and approval gates before tool execution, 2025. Cited for comparative positioning.
- [3] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. Accessed 2025.

- [4] Arnold Cartagena and Ariane Teixeira. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents. *arXiv preprint arXiv:2602.16943*, 2026.
- [5] Matt Howard. The Envoy proxy: A communication bus for microservices. In *USENIX ;login:*, 2019.
- [6] Minghao Li, Yixin Zhao, Bowen Wang, Peixin Lai, Hongsheng Wang, and Yujiu Cao. Api-bank: A comprehensive benchmark for tool-augmented llms. In *EMNLP*, 2023.
- [7] MCPSHIELD Authors. A formal security framework for MCP-based AI agents: Threat taxonomy, verification models, and defense mechanisms. *arXiv preprint arXiv:2604.05969*, 2026. Formal threat taxonomy and defense-in-depth reference architecture.
- [8] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Chen. Gorilla: Large language model connected with massive apis. In *arXiv preprint arXiv:2305.15334*, 2023.
- [9] Progent Authors. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025.
- [10] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxin Lu, Yujiu Lin, Xin Cong, Xiangru Tang, Qian Yang, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024.
- [11] Leonard Richardson and Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2007.
- [12] SEAgent Authors. Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework. *arXiv preprint arXiv:2601.11893*, 2026.
- [13] Aojie Yuan, Zhiyuan Su, and Yue Zhao. AEGIS: No tool call left unchecked: A pre-execution firewall and audit layer for AI agents. *arXiv preprint arXiv:2603.12621*, 2026.